SRB — Guía rápida del código (primera entrega)

Oct 15, 2026 · @Juan Sebastian Moreno Cortes

Estado actual

El cliente, el Gestor de Transacciones (GT) y el RMC están escritos. **Falta la capa de Persistencia con la base de datos**, y sin ella ni la consulta ni la reserva funcionan de punta a punta. Nada se ha probado todavía con los cuatro procesos juntos.

Pieza	Archivo	Estado
Contrato de mensajes	`comun/mensajes.py`	Hecho, con errores conocidos (ver abajo)
Configuración	`comun/config.py` + `.env`	Hecho; `DB_URL` está vacío (`*`)
Cliente	`capa_consumidores/cliente.py`	Hecho: consulta y reserva
Generador de carga	`capa_consumidores/carga_clientes.py`	Hecho: 20 peticiones/min, 60 % consultas (fijo en el código)
Arranque de consumidores	`capa_consumidores/main.py`	Modos `caso` y `carga`
Gestor de Transacciones	`capa_servicios/gestor_de_transacciones.py`	Hecho: ROUTER + DEALER con Poller
Servicio RMC	`capa_servicios/servicio_rmc.py`	Consulta y reserva hechas; modificación responde NO_IMPLEMENTADO
Arranque de servicios	`capa_servicios/main.py`	Arranca `gt` o `rmc` según el argumento
Cancelación y notificación	`cancelacion_y_notificacion.py`	Vacío (segunda entrega)

Pieza	Archivo	Estado
Persistencia + BD	—	**Por hacer**

Verifiquen el nombre real de la carpeta de servicios: los imports de `capa_servicios/main.py` asumen `capa_servicios`.

Estructura y cómo ejecutar

Cada capa es una carpeta, y **cada proceso tiene un solo punto de arranque**. Todo se ejecuta **desde la raíz del proyecto con `python -m`**; si ejecutan un archivo por su ruta (`python capa_servicios/main.py`), falla el `import comun`.

```
srb/
├── .env                    # IPs y puertos (no se sube a git)
├── comun/
│   ├── mensajes.py         # EL CONTRATO entre procesos
│   └── config.py           # lee el .env
├── capa_consumidores/
│   ├── cliente.py          # clase Cliente (socket REQ)
│   ├── carga_clientes.py   # genera la carga
│   └── main.py             # arranque: caso | carga
├── capa_servicios/
│   ├── gestor_de_transacciones.py
│   ├── servicio_rmc.py     # interfaz + Servicio_RMC +
Cliente_persistencia
│   ├── cancelacion_y_notificacion.py
│   └── main.py             # arranque: gt | rmc
└── capa_almacenamiento/    # POR HACER
```

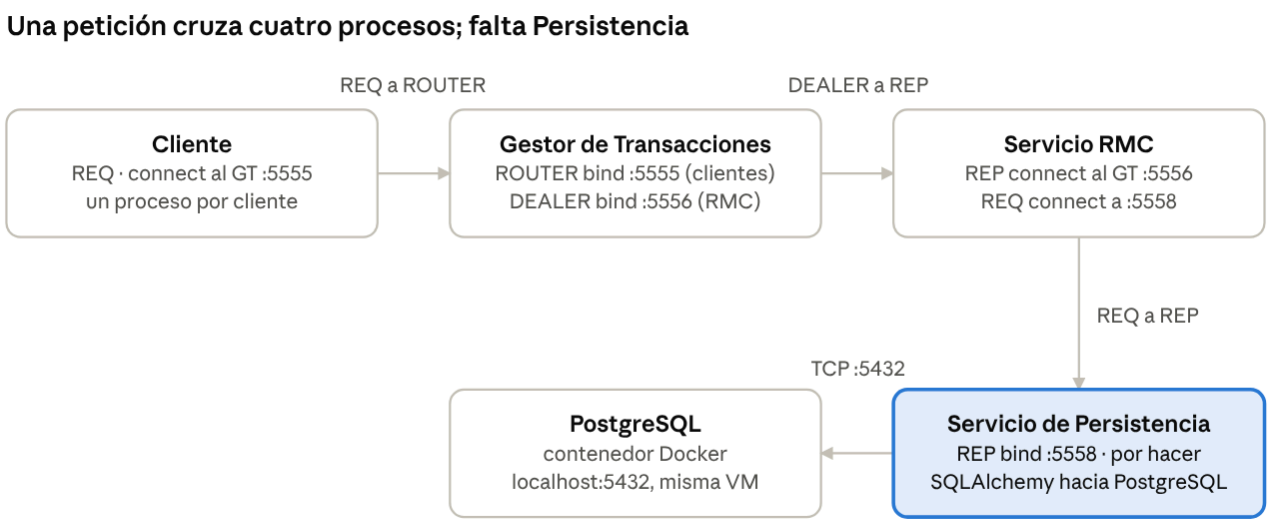
Cada carpeta necesita un `__init__.py` (puede estar vacío).

Comandos, cada uno en su terminal:

```
python -m capa_servicios.main gt
python -m capa_servicios.main rmc
python -m capa_consumidores.main caso
```

El `.env` vive en la raíz. Para desarrollo local los tres `HOST_` van en `localhost`; en las VM, la IP de cada máquina. Los puertos actuales: GT clientes `5555`, GT→RMC `5556`, GT→Cancelación `5557`, Persistencia `5558`. No suban el `.env` (tendrá la contraseña de la BD): suban un `.env.example` con valores de ejemplo.

Cómo viaja una petición



camino de una petición · 4 procesos y la BD

Las respuestas vuelven por el mismo camino. El ROUTER del GT le pega a cada petición la identidad del cliente, y el GT reenvía los frames sin tocarlos. El REP del RMC guarda ese encabezado y lo vuelve a pegar al responder; así el ROUTER sabe a qué cliente devolver la respuesta.

Los sockets estables hacen `bind` (los dos del GT y el de Persistencia); clientes y RMC hacen `connect`. Gracias a eso, en la segunda entrega se pueden agregar réplicas del RMC sin tocar el GT: el DEALER les reparte el trabajo solo.

El contrato: `comun/mensajes.py`

Entre procesos no hay llamadas a métodos, solo mensajes JSON por ZeroMQ. `mensajes.py` es la única definición de esos mensajes. Si un proceso usa otro nombre de campo u otro valor, nada falla al arrancar: el mensaje simplemente no se entiende y el cliente se queda esperando. **No inventen campos ni valores: si falta algo, se agrega aquí y se avisa al equipo.**

Petición (`Peticion`): `uuid` (texto, único por petición), `tipo` (texto), `id_cliente` (entero), `datos` (diccionario).

Respuesta (`Respuesta`): `uuid` (el MISMO de la petición que responde), `estado` , `motivo` , `datos` .

Abreviatura	Valor que viaja	Para qué
`CO`	`CONSULTA`	Cliente → RMC → Persistencia: eventos de un mes
`R`	`RESERVA`	Cliente → RMC
`M`	`MODIFICACION`	Cliente → RMC (no implementada)
`CA`	`CANCELACION`	Segunda entrega
`N`	`NOTIFICACION`	Segunda entrega
`CO_D`	`CONSULTA_DISPONIBILIDAD`	RMC → Persistencia
`A_R`	`AGREGAR_RESERVA`	RMC → Persistencia

Estados	Motivos
`E` = `EXITO`	`S_C` = `SIN_CUPO`
`F` = `FALLO` (la operación no se pudo hacer)	`N_E` = `NO_EXISTE`
`ER` = `ERROR` (algo falló en el sistema)	`D_I` = `DATO_INVALIDO`
`A` = `ACK`	`P_I` = `PETICION_INVALIDA`, `N_I` = `NO_IMPLEMENTADO`

Reglas que no se pueden romper:

- **Lo que viaja es el valor completo** (`"EXITO"`, `"RESERVA"`). Las abreviaturas solo se usan como argumento de `crear_req` y `crear_rta`. Cuidado: `R` es RESERVA en peticiones, y en estados no existe.
- **Codificar y decodificar solo en los bordes**: `codificar` = `json.dumps`, `decodificar` = `json.loads`. Dentro de los servicios todo es diccionario. Solo el `main` de cada proceso y `Cliente_persistencia` convierten a texto.
- **Toda respuesta lleva la clave `estado`.** El RMC decide con `respuesta['estado'] == 'EXITO'`.
- **Fechas como texto**, nunca como objeto `date` (no se puede convertir a JSON).

Contrato con Persistencia (lo que hay que respetar al construirla)

Persistencia es un proceso con un socket REP que hace `bind` en `tcp://*:5558` (`SP_PUERTO`). El RMC se conecta con un REQ, envía texto JSON (`send_string`) y espera exactamente una respuesta por cada petición. Si Persistencia recibe y no responde, el RMC y todos los clientes quedan colgados: el ciclo debe ir dentro de `try/except` y responder `ERROR` si algo falla.

El RMC ya está programado para enviar y leer exactamente esto:

`tipo` que llega	`datos` que llegan	Qué hace Persistencia	Qué debe responder
`CONSULTA`	`{"mes": "noviembre"}`	Busca los eventos con ocurrencias en ese mes	`EXITO` + la lista de eventos (formato por acordar, ver abajo)
`CONSULTA_DISPONIBILIDAD`	`{"id_evento": "E1", "fecha": "2026/11/07"}`	Busca la ocurrencia de ese evento en esa fecha	`EXITO` + `{"capacidad_total": 100, "ocupacion": 40}`; si no existe, `FALLO` + `NO_EXISTE`
`AGREGAR_RESERVA`	`{"id_evento": "E1", "fecha": "2026/11/07", "n_asientos": 2}`	Suma los puestos **solo si caben** e inserta la reserva, en una sola transacción	`EXITO` + `{"id_reserva": 17}`; si no caben, `FALLO` + `SIN_CUPO`

Detalles que causan errores silenciosos si no se respetan:

- **Nombres exactos de las claves de respuesta: `capacidad_total` y `ocupacion`.** El RMC lee esas dos. Aunque en la BD las columnas se llamen `capacidad` y `puestos_ocupados`, la respuesta debe usar estos nombres.
- **La fecha llega con barras: `2026/11/07`.** Se convierte con `datetime.strptime(fecha, '%Y/%m/%d').date()`. `date.fromisoformat` falla con barras.
- **El mes llega como texto en español y minúsculas.** Su número es su posición en la lista `meses` de `mensajes.py` más 1.
- **`id_evento` es texto** (`"E1"`), no número.
- **La respuesta lleva el mismo `uuid` que la petición.** Así se puede seguir una petición en los logs.

- **La `CONSULTA` llega tal cual la envió el cliente**: el RMC la reenvía sin cambios y devuelve al cliente lo que responda Persistencia.

Por qué `AGREGAR_RESERVA` debe verificar el cupo otra vez, aunque el RMC ya consultó la disponibilidad: entre las dos llamadas otro cliente puede reservar. La forma segura es un `UPDATE ... SET puestos_ocupados = puestos_ocupados + n WHERE ... AND puestos_ocupados + n <= capacidad` y revisar `rowcount`: si es 0, no había cupo. Esto se explica con un simulador en el tutorial de SQLAlchemy.

Pendiente de acordar: el formato de la lista de eventos en `CONSULTA`. Propuesta: `{"eventos": [{"id_evento", "nombre", "tipo_evento", "fecha", "capacidad", "puestos_ocupados"}]}`, con la fecha en el mismo formato con barras. Quien haga Persistencia lo define y lo anota aquí.

Errores conocidos que hay que corregir

Estos errores se encontraron revisando el código, no ejecutándolo. Algunos pueden estar ya corregidos: verifiquen antes de tocar. Los tres primeros rompen el flujo.

- [ ] **`mensajes.crear_req` tiene la condición al revés.** Cuando le pasan un uuid, genera uno nuevo; cuando no le pasan ninguno (el cliente), usa `""`. Debe generar uno nuevo solo si viene vacío.
- [ ] **`mensajes.crear_rta` no recibe el uuid de la petición** y genera uno nuevo, así que el cliente no puede emparejar respuestas. Además exige un motivo válido, por lo que **no se puede crear una respuesta de `EXITO`**. Persistencia la necesita: hagan el motivo opcional.
- [ ] **`servicio_rmc.crear_reserva` no debe decodificar la petición.** El `main` del RMC ya la decodifica; si queda un `decodificar(peticion)` al inicio, lanza `TypeError`.
- [ ] **El RMC responde `ER` (ERROR) para SIN_CUPO y NO_EXISTE.** Debe ser `F` (FALLO): no tener cupo no es un error del sistema.
- [ ] **`Cliente_persistencia` no tiene timeout.** Si Persistencia no responde, el RMC se bloquea para siempre.
- [ ] **`carga_clientes.py` llama a `main_carga()` sin `if __name__ == "__main__":`.** Si otro archivo lo importa, arranca la carga sola.
- [ ] **La carga reserva fechas al azar de enero y consulta meses al azar.** Contra la BD real casi todo dará NO_EXISTE. Usen fechas y eventos que existan en los datos cargados.
- [ ] **`.env`: `DB_URL=*`.** Con PostgreSQL en contenedor: `postgresql+psycopg://srb:srb@localhost:5432/srb`. `DB_PASSWD` sobra (la contraseña va dentro de la URL).

- [ ] **El cliente no atrapa el timeout del socket** fuera del `main`. Después de un timeout, ese socket REQ queda inservible y hay que recrearlo.

Cómo probar de punta a punta

Cada proceso va en su propia terminal, siempre desde la raíz. En ZeroMQ el orden de arranque no importa, pero este orden hace más fácil ver dónde falla algo.

1. Levantar PostgreSQL en su contenedor y cargar los datos iniciales.
2. Arrancar Persistencia.
3. `python -m capa_servicios.main gt`
4. `python -m capa_servicios.main rmc`
5. `python -m capa_consumidores.main caso`

Antes de tener la Persistencia real, prueben con una **Persistencia falsa**: un socket REP con `bind` en el puerto 5558 que responde datos fijos según el `tipo` (dos eventos inventados para `CONSULTA`, `{"capacidad_total": 100, "ocupacion": 0}` para disponibilidad, `EXITO` con un id para agregar). Así se prueba la cadena completa sin base de datos.

Cómo leer lo que pasa:

- **El cliente muestra "Timeout" a los 5 segundos**: algún proceso de la cadena no respondió. Revisen cuál imprimió la petición y cuál no.
- **El GT imprime la petición pero no la respuesta**: el problema está en el RMC o en Persistencia.
- **"Address already in use"**: otro proceso ya tiene ese puerto. Probablemente quedaron procesos viejos corriendo.
- **`KeyError` en el RMC**: Persistencia respondió con un nombre de clave distinto al del contrato.

Decisiones abiertas del equipo

Ninguna bloquea la primera entrega, pero el informe y el código deben decir lo mismo antes de la sustentación.

- [ ] **Formato de la respuesta de `CONSULTA`.** Lo define quien haga Persistencia y lo anota en la sección del contrato.
- [ ] **PostgreSQL en Docker en la VM de almacenamiento.** Instalar Docker requiere permisos de administrador; prueben pronto si la VM lo permite. Si no, el plan B es instalar PostgreSQL directo o usar SQLite para la primera entrega (el enunciado dice "la BD o archivo").

- [ ] **Patrón hacia Persistencia.** El código usa REQ (RMC) → REP (Persistencia). El informe dice REQ/ROUTER. Corrijan uno de los dos.
- [ ] **Generación de carga.** El informe dice Locust; el código tiene un generador propio. Escojan uno y actualicen el informe.
- [ ] **Informe, sección 1.1.** Dice que las cancelaciones no notifican al cliente; el enunciado exige notificarlo.
- [ ] **Diagramas.** El de despliegue y el de secuencia ya están en Lucidchart. En el de clases faltan el `ID` de `Ocurrencia` y los sockets como atributos del GT.

Para aprender SQLAlchemy desde cero (Core y ORM, con PostgreSQL en contenedor) hay un tutorial interactivo aparte; el enlace se agrega aquí.